Übung 4: Hashing und Hashtabellen

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, Hashtabellen mit den Kollisionsstrategien Verkettung, lineares Sondieren, quadratisches Sondieren und doppeltes Hashing manuell zu befüllen und Suchoperationen darauf nachzuvollziehen. Sie üben, für eine gegebene Problemstellung einen geeigneten abstrakten Datentyp und eine passende Datenstruktur auszuwählen und deren Laufzeitverhalten in O-Notation zu begründen. Zusätzlich lernen Sie anhand des Geburtstagsparadoxons, wie man die Kollisionswahrscheinlichkeit in einer Hashtabelle formal herleitet und berechnet.

1. Hashtabelle: Hashing mit Verkettung
   (a) Nehmen wir an, wir haben eine Hashtabelle, die durch Hashing mit Verkettung implementiert wurde. Diese Hashtabelle hat eine interne Kapazität von 10. Die Slots werden durch eine verkettete Liste implementiert, an deren Ende neue Elemente angehängt werden. Die Größe der Hashtabelle bleibt unverändert. Zeigen Sie, wie diese Hashtabelle intern aussieht, nachdem die folgenden Schlüssel-Wert-Paare in der angegebenen Reihenfolge unter Verwendung der Hashfunktion $h(x) = (4x) \mod 10$ eingefügt wurden:

   $$(1, a), (4, b), (2, c), (17, d), (12, e), (9, e), (19, f), (4, g), (8, c), (12, f)$$

   (b) Betrachten Sie das folgende Szenario: Angenommen, wir haben eine Hashtabelle mit einer anfänglichen Kapazität von 12. Wir verändern die Größe der Hashtabelle, indem wir die Kapazität verdoppeln. Angenommen, wir fügen ganzzahlige Schlüssel in diese Tabelle ein, indem wir die Hashfunktion

   $$h(x) = (4x) \mod \text{Kapazität}$$

   verwenden: Warum ist diese Wahl der Hashfunktion und der Anfangskapazität suboptimal? Wie können wir sie korrigieren?

2. Folgende Schlüssel $10, 22, 31, 4, 15, 28, 17, 88$ und 59 sollen in eine Hashtabelle der Größe $m = 11$ mit offener Adressierung eingefügt werden. Benutzen Sie als primäre Hashfunktion $h(k) = k \mod m$.

   Wie lauten die Ergebnisse für
   (a) Lineare Sondierung.
   (b) Quadratische Sondierung mit $c_1 = 1, c_2 = 3$.
   (c) Doppeltes Hashing mit $h_1(k) = k \mod m$ und $h_2(k) = k \mod (m - 1)$.

3. Betrachten Sie die folgenden Schlüssel-Wert-Paare:

   $$(6, a), (29, b), (41, d), (34, e), (10, f), (64, g), (50, h)$$

   Die Hashfunktion lautet $h(k) = 2k \mod \text{tablesize}$. Fügen Sie jedes Schlüssel-Wert-Paar in die folgenden Hashtabellen ein und zeichnen Sie ein, wie ihr interner Zustand aussieht:
   (a) Eine Hashtabelle, die Hashing mit Verkettung verwendet. Die Tabelle hat eine interne Kapazität von 10. Gehen Sie davon aus, dass jeder Slot eine verkettete Liste ist, an deren Ende neue Paare angehängt werden. Um eine Größenänderung müssen Sie sich nicht kümmern.
   (b) Eine Hashtabelle, die lineares Sondieren verwendet, mit einer internen Kapazität von 10. Um eine Größenänderung müssen Sie sich nicht kümmern.
   (c) Eine Hashtabelle, die quadratisches Sondieren verwendet, mit einer internen Kapazität von 10 und $c_1 = 0, c_2 = 1$. Um eine Größenänderung müssen Sie sich nicht kümmern.

4. Hashing mit offener Adressierung: Zur Speicherung von Schlüsseln (k) in einer Hashtabelle der Größe 11 soll folgende doppelte Hashfunktion verwendet werden.

$$h(k, i) = (k \bmod 11 + i \cdot (1 + (k \bmod 5))) \bmod 11$$

   (a) Tragen Sie die folgenden Schlüssel (k) in der angegebenen Reihenfolge in eine anfangs leere Hashtabelle ein. Zeichnen Sie dazu die Hashtabelle und tragen Sie die Schlüssel in die korrekten Indizes der Hashtabelle ein.

$$10, 20, 30, 40, 19$$

   (b) Geben Sie an, wie viele Zugriffe auf die Hashtabelle man für die erfolglose Suche nach dem Schlüssel 7 benötigt. Auf welche Indizes der Hashtabelle wird zugegriffen?

5. (Über Klausurniveau) Kollisionen kommen häufiger vor, als man intuitiv vermutet. Dieses Phänomen ist als das Geburtstagsparadoxon (nach R. von Mises) bekannt: Befinden sich in einem Raum mindestens 23 Personen, dann ist die Wahrscheinlichkeit, dass zwei oder mehr Personen am selben Tag Geburtstag haben, größer als 50 %.

   Lit.: R. von Mises. Über Aufteilungs- und Besetzungs-Wahrscheinlichkeiten. Revue de la Faculté des Sciences de l'Université d'Istanbul, N.S. 4. 1938-39, S. 145-63

   Übertragen auf das Kollisionsproblem bedeutet dies:

   Bei einer Hashtabelle T mit 365 Slots und 23 zu speichernden Schlüsseln im Wertebereich $[0, 364]$ ist die Wahrscheinlichkeit einer Kollision größer als 50 %.

   Berechnen Sie die Wahrscheinlichkeit P(n) für ein oder mehrere Kollisionen, wenn man n Schlüssel in eine Hashtabelle T der Größe 365 einfügt.

   Programmieren Sie die Funktion P(n), und geben Sie in einem Koordinatensystem die Ergebnisse (Wahrscheinlichkeiten) aus für n = 0, 5, 10, ..., 80 Schlüssel (x-Achse). Lesen Sie den Wert für n=23 aus der Kurve ab.

   Lösungshinweis:

   Sei Q(n) die Wahrscheinlichkeit, dass n Schlüssel zufällig in einer Hashtabelle T der Größe m=365 gespeichert werden und keine Kollision auftritt. Sei P(n) die Wahrscheinlichkeit für mindestens eine Kollision. Dann gilt:

$$P(n) = 1 - Q(n)$$

   da 1 minus die Wahrscheinlichkeit für keine Kollision gleich der Wahrscheinlichkeit für mindestens eine Kollision ist.

   Es gilt: Q(1) = 1. Überlegen Sie sich Q(2), Q(3), ... Sie erhalten eine Differenzgleichung, die Sie durch Entfaltung lösen können.

6. Sie arbeiten an einer Anwendung, die eine Sammlung von Büchern verwalten soll. Jedes Buch hat eine eindeutige ISBN-Nummer, einen Titel, einen Autor und ein Veröffentlichungsdatum. Die Anwendung soll in der Lage sein, die folgenden vier Operationen effizient auszuführen:

   1. Einfügen eines neuen Buches mit allen seinen Daten.
   2. Suchen eines Buches anhand der ISBN.
   3. Aktualisieren des Autors oder des Veröffentlichungsdatums eines Buches.
   4. Löschen eines Buches aus der Sammlung.

   Wählen Sie einen geeigneten abstrakten Datentyp und eine Datenstruktur, um die Buchdaten effizient zu verwalten und die oben genannten Operationen optimal durchzuführen. Begründen Sie Ihre Wahl und erläutern Sie, warum dieser abstrakte Datentyp und diese Datenstruktur geeignet sind. Geben Sie zusätzlich die erwarteten Laufzeiten für die vier oben genannten Operationen in der O-Notation an.

7. Sie arbeiten an einer Anwendung, die die Aufgabenverwaltung für ein Team vereinfachen soll. Jedes Teammitglied hat eine eindeutige ID, einen Namen und eine Liste von Aufgaben, die ihm zugewiesen sind. Die Anwendung soll in der Lage sein, die folgenden vier Operationen effizient auszuführen:

   1. Einfügen eines neuen Teammitglieds mit all seinen Daten, einschließlich der Liste der zugewiesenen Aufgaben.
   2. Suchen eines Teammitglieds anhand seiner ID.
   3. Ersetzen der Liste der zugewiesenen Aufgaben für ein Teammitglied durch eine neue Liste.
   4. Löschen eines Teammitglieds aus der Aufgabenverwaltung.

   Wählen Sie einen geeigneten abstrakten Datentyp und eine Datenstruktur, um die Teammitglieder und ihre Aufgaben effizient zu verwalten und die oben genannten Operationen optimal durchzuführen. Begründen Sie Ihre Wahl und erläutern Sie, warum dieser abstrakte Datentyp und diese Datenstruktur geeignet sind. Geben Sie zusätzlich die erwarteten Laufzeiten für die vier oben genannten Operationen in der O-Notation an.

8. Hashing mit offener Adressierung: Zur Speicherung von Zeichenketten in einer Hashtabelle der Größe 11 soll folgende Hashfunktion `hash_code` verwendet werden. Bei Kollisionen soll lineare Sondierung genutzt werden. Hinweis: Der ASCII-Wert des Zeichens “a” beträgt 97, “b” entsprechend 98, etc. Dieser Wert wird von der Funktion `ord` zurückgegeben.

```
def hash_code(input_string):
  output = 0
  for c in input_string:  # for each character c in the input_string
    output += ord(c) - 90
  return output % 11
```

   (a) Tragen Sie die folgenden Zeichenketten in der angegebenen Reihenfolge in eine anfangs leere Hashtabelle ein. Tragen Sie dazu die Zeichenketten in die korrekten Indizes der gegebenen Hashtabelle ein. Bitte geben Sie den Rechenweg an.
   Zeichenketten: “abc”, “ac”, “bc”, “ca”

0	1	2	3	4	5	6	7	8	9	10

   (b) Geben Sie an, wie viele Zugriffe auf die Hashtabelle für die erfolglose Suche nach dem Schlüssel “ccc” benötigt werden. Auf welche Indizes der Hashtabelle wird zugegriffen?

9. Hashing mit Verkettung: Zur Speicherung von Zeichenketten in einer Hashtabelle der Größe 11 soll folgende Hashfunktion `hash_code` verwendet werden. Bei Kollisionen soll Hashing mit Verkettung genutzt werden. Hinweis: Der ASCII-Wert des Zeichens “a” beträgt 97, “b” entsprechend 98, etc. Dieser Wert wird von der Funktion `ord` zurückgegeben.

```
def hash_code(input_string):
  output = 1
  for c in input_string:  # for each character c in the input_string
    output *= ord(c) - 95
  return output % 11
```

   (a) Tragen Sie die folgenden Zeichenketten in der angegebenen Reihenfolge in eine anfangs leere Hashtabelle ein. Tragen Sie dazu die Zeichenketten in die korrekten Slots der gegebenen Hashtabelle ein. Bitte geben Sie den Rechenweg an.
   Zeichenketten: “abc”, “ac”, “cba”, “acb”, “ca”

0	1	2	3	4	5	6	7	8	9	10

(b) Geben Sie an, wie viele Operationen auf der Hashtabelle für die erfolglose Suche nach dem Schlüssel "cab" ausgeführt werden müssen. Auf welche Slots der Hashtabelle wird zugegriffen?

10. Löschen bei offener Adressierung

Bei offener Adressierung kann ein Slot nicht einfach als "leer" markiert werden, wenn ein Schlüssel gelöscht wird, da dies die Sondierungskette für andere Schlüssel unterbrechen würde. Stattdessen wird häufig ein spezieller Marker (*Tombstone* bzw. *DELETED*) verwendet.

(a) Erklären Sie anhand eines kurzen Beispiels, warum das einfache Markieren eines Slots als "leer" (statt als *Tombstone*) beim Löschen zu einem fehlerhaften Suchergebnis führen kann.

(b) Gegeben sei die Hashtabelle aus Aufgabe 2a) (lineares Sondieren, $m = 11$, $h(k) = k \mod 11$) nach dem Einfügen aller neun Schlüssel. Angenommen, der Schlüssel 17 wird nun gelöscht und durch einen *Tombstone* ersetzt. Zeichnen Sie den resultierenden Zustand der Hashtabelle.

(c) Beschreiben Sie, wie sich die erfolgreiche Suche nach dem Schlüssel 59 nach diesem Löschvorgang verhält (Rechenweg mit Indizes angeben). Muss die Suche an einem *Tombstone*-Eintrag fortgesetzt oder abgebrochen werden? Begründen Sie.

11. Belegungsfaktor und Wahl der Kollisionsstrategie

(a) Geben Sie den Belegungsfaktor der Hashtabelle aus Aufgabe 2 nach dem Einfügen aller neun Schlüssel an.

(b) Bei offener Adressierung muss stets $\lambda \leq 1$ gelten, bei Hashing mit Verkettung hingegen nicht. Erklären Sie, warum dieser Unterschied besteht.

(c) Nennen Sie für einen sehr hohen Belegungsfaktor (λ nahe 1) jeweils einen Vor- bzw. Nachteil von Hashing mit Verkettung im Vergleich zu offener Adressierung.

Dieses Übungsblatt ist Teil der Lehrveranstaltung *„Algorithmik"*.